Digital Clock Project

Complete Documentation

Components Used

ATmega328P (Arduino Uno) · DS1307 RTC · 7-Segment Displays

Displays time in **12-hour format** with AM/PM indicator

Last updated: February 2025

Contents

1 Overview **2**

2 Hardware Requirements **2**

3 Wiring Diagram **2**
3.1 DS1307 RTC to Arduino Uno . . . 2

4 Software Setup **3**
4.1 Required Libraries . . . 3

5 Step 1: I2C Scanner **3**
5.1 Purpose . . . 3
5.2 When to Use . . . 3
5.3 Code . . . 3
5.4 How to Use . . . 4
5.5 Expected Results . . . 4
5.6 Troubleshooting . . . 4

6 Step 2: Save Time **4**
6.1 Purpose . . . 4
6.2 When to Use . . . 4
6.3 Code . . . 5
6.4 24-Hour to 12-Hour Conversion Reference . . . 6
6.5 Step-by-Step Process . . . 6

7 Step 3: Digital Clock (Main Program) **7**
7.1 Purpose . . . 7
7.2 Features . . . 7
7.3 Pin Mapping . . . 7
7.4 Code . . . 7
7.5 Segment Encoding Reference . . . 10

8 AM/PM Indicator Logic **10**

9 Quick-Start Checklist **11**

1. Overview

This project creates a digital clock using an **ATmega328P (Arduino Uno)**, a **DS1307 RTC module**, and **7-segment displays**. The clock shows time in 12-hour format with an AM/PM indicator driven by the decimal-point (DP) segment of the first digit.

The project is organised into three sequential steps:

1. **I2C Scanner** — verify the RTC module is wired correctly
2. **Save Time** — set the current time into the DS1307
3. **Digital Clock** — display the time on the 7-segment displays

2. Hardware Requirements

Component	Quantity	Description
Arduino Uno (ATmega328P)	1	Main microcontroller
DS1307 RTC Module	1	Real-time clock chip
7-Segment Display (Common Anode)	4	For displaying time digits
CR2032 Battery	1	RTC backup power
Resistors (220 Ω)	7	Segment current limiting
Jumper Wires	Several	For connections

Table 1: Bill of Materials

3. Wiring Diagram

3.1. DS1307 RTC to Arduino Uno

DS1307 Pin		Arduino Uno Pin
VCC	⟶	5V
GND	⟶	GND
SDA	⟶	A4
SCL	⟶	A5

Table 2: I2C wiring for DS1307

Note: The DS1307 communicates over the **I2C bus**. On the Arduino Uno, SDA is on pin A4 and SCL is on pin A5. These are the only two wires needed for data communication (plus power).

4. Software Setup

4.1. Required Libraries

1. **Wire.h** — Built-in Arduino I2C library (no installation needed)
2. **RTClib.h** — Install via Arduino Library Manager:
   - Go to: `Sketch` → `Include Library` → `Manage Libraries`
   - Search: `RTClib`
   - Install: `RTClib by Adafruit`

5. Step 1: I2C Scanner

5.1. Purpose

Verify that the DS1307 module is properly connected and responding on the I2C bus.

5.2. When to Use

- First-time setup
- Troubleshooting connection issues
- Verifying wiring after changes

5.3. Code

```
#include <Wire.h>

void setup(){
  Wire.begin();
  Serial.begin(9600);
  Serial.println("Scanning I2C devices...");

  byte count = 0;
  for(byte address = 1; address < 127; address++){
    Wire.beginTransmission(address);
    byte error = Wire.endTransmission();

    if(error == 0){
      Serial.print("I2C device found at address 0x");
      if(address < 16)
        Serial.print("0");
      Serial.println(address, HEX);
      count++;
    }
  }

  if(count == 0)
    Serial.println("No I2C devices found! Check your wiring!");
  else
    Serial.println("Scan complete!");
}

```

```
void loop(){}
```

Listing 1: I2C Scanner

5.4. How to Use

1. Upload the code to your Arduino
2. Open Serial Monitor (`Ctrl+Shift+M`)
3. Set baud rate to `9600`

5.5. Expected Results

SUCCESS:

```
Scanning I2C devices...
I2C device found at address 0x68
Scan complete!
```

FAILURE:

```
Scanning I2C devices...
No I2C devices found! Check your wiring!
```

5.6. Troubleshooting

If the scan fails, check the following connections:

- SDA → A4
- SCL → A5
- VCC → 5V
- GND → GND
- Try a different DS1307 module if the above are correct

6. Step 2: Save Time

6.1. Purpose

Set the current time in the DS1307 RTC chip so it can begin counting.

6.2. When to Use

- Initial setup
- After power loss (if no backup battery)
- To correct time drift
- When changing time zones

6.3. Code

```
#include "Wire.h"
#define DS1307_ADDRESS 0x68
byte zero = 0x00;

void setup(){
  Wire.begin();
  Serial.begin(9600);
  setDateTime();
  printDate();
  Serial.println("Time Set Successfully!");
}

void loop(){
  printDate();
  delay(1000);
}

void setDateTime(){

  // ============================
  // ONLY CHANGE THESE 2 VALUES:
  // ============================
  byte minute =  30;  // <- Current minutes (0-59)
  byte hour   =  14;  // <- Current hour in 24h format (0-23)
  // ============================

  // DO NOT CHANGE BELOW THIS LINE
  byte second  = 0;
  byte weekDay = 6;   // Friday
  byte monthDay= 14;  // 14th
  byte month   = 2;   // February
  byte year    = 25;  // 2025

  Wire.beginTransmission(DS1307_ADDRESS);
  Wire.write(zero);

  Wire.write(decToBcd(second));
  Wire.write(decToBcd(minute));
  Wire.write(decToBcd(hour));
  Wire.write(decToBcd(weekDay));
  Wire.write(decToBcd(monthDay));
  Wire.write(decToBcd(month));
  Wire.write(decToBcd(year));

  Wire.write(zero);
  Wire.endTransmission();
}

byte decToBcd(byte val){
  return ( (val/10*16) + (val%10) );
}

byte bcdToDec(byte val){
  return ( (val/16*10) + (val%16) );
}

```

```
void printDate(){
  Wire.beginTransmission(DS1307_ADDRESS);
  Wire.write(zero);
  Wire.endTransmission();

  Wire.requestFrom(DS1307_ADDRESS, 7);

  int second  = bcdToDec(Wire.read());
  int minute  = bcdToDec(Wire.read());
  int hour    = bcdToDec(Wire.read() & 0b111111);
  int weekDay = bcdToDec(Wire.read());
  int monthDay= bcdToDec(Wire.read());
  int month   = bcdToDec(Wire.read());
  int year    = bcdToDec(Wire.read());

  // Convert to 12-hour format
  String ampm = "AM";
  int hour12  = hour;

  if(hour == 0){
    hour12 = 12; ampm = "AM";
  } else if(hour < 12){
    hour12 = hour; ampm = "AM";
  } else if(hour == 12){
    hour12 = 12; ampm = "PM";
  } else {
    hour12 = hour - 12; ampm = "PM";
  }

  Serial.print(month);   Serial.print("/");
  Serial.print(monthDay); Serial.print("/");
  Serial.print(year);    Serial.print("     ");
  Serial.print(hour12);  Serial.print(":");
  if(minute < 10) Serial.print("0");
  Serial.print(minute);  Serial.print(":");
  if(second < 10) Serial.print("0");
  Serial.print(second);  Serial.print(" ");
  Serial.println(ampm);
}
```

Listing 2: Save Time / Set RTC

6.4. 24-Hour to 12-Hour Conversion Reference

Important: Always use **24-hour format** when entering the `hour` value.

6.5. Step-by-Step Process

1. Check the current time (use your phone or computer clock)
2. Edit these two lines in the code:

```
byte minute =  14;  // <- Your current minutes
byte hour   =  18;  // <- Your current hour (24h format)
```

Desired Time	Hour Value	Minute Value
12:00 AM	0	0
6:30 AM	6	30
9:15 AM	9	15
12:00 PM	12	0
1:45 PM	13	45
5:30 PM	17	30
6:14 PM	18	14
9:00 PM	21	0
11:59 PM	23	59

Table 3: 24-hour format reference table

3. Upload the code to the Arduino
4. Open Serial Monitor at `9600` baud
5. Verify the output looks like:

```
Time Set Successfully!
2/14/25      6:14:00 PM
2/14/25      6:14:01 PM
```

Tip: Upload takes approximately 5 seconds. Add 5–10 seconds to your target time to compensate for upload delay, keeping the clock accurate.

7. Step 3: Digital Clock (Main Program)

7.1. Purpose

Display the time on the four 7-segment displays in 12-hour format.

7.2. Features

- **12-hour format** display
- **Hides leading zero** — "9:30" shows as `9 30`, not `09 30`
- **AM/PM indicator** via the DP (decimal point) on digit 1
- **Blinking colon** — toggles every second using digit 2's DP

7.3. Pin Mapping

7.4. Code

Segment	Arduino Pin	Common Anode	Arduino Pin
A	2	CA1 (Digit 1)	10
B	3	CA2 (Digit 2)	11
C	4	CA3 (Digit 3)	12
D	5	CA4 (Digit 4)	13
E	6		
F	7		
G	8		
DP	9		

Table 4: 7-Segment display pin assignments

```
#include <Wire.h>
#include "RTClib.h"

#define A 2
#define B 3
#define C 4
#define D 5
#define E 6
#define F 7
#define G 8
#define DP 9

#define CA1 10
#define CA2 11
#define CA3 12
#define CA4 13

int d = 2;
int first_h=0, second_h=0, first_m=0, second_m=0;
int h, m, s, h12;
bool isPM = false;
RTC_DS1307 rtc;

const int segs[7] = { A, B, C, D, E, F, G };

// Segment patterns for digits 0-9 (Common Anode, active LOW)
const byte numbers[10] = {
  0b1000000, // 0
  0b1111001, // 1
  0b0100100, // 2
  0b0110000, // 3
  0b0011001, // 4
  0b0010010, // 5
  0b0000010, // 6
  0b1111000, // 7
  0b0000000, // 8
  0b0010000  // 9
};

```

```
const byte blank = 0b1111111;

void setup(){
  Serial.begin(57600);
  Wire.begin();
  rtc.begin();

  for(int i = 2; i <= 13; i++){
    pinMode(i, OUTPUT);
    digitalWrite(i, LOW);
  }
}

void loop(){
  DateTime now = rtc.now();
  h = now.hour();
  m = now.minute();
  s = now.second();

  // Convert 24h to 12h
  if(h == 0)        { h12 = 12; isPM = false; }
  else if(h < 12)   { h12 = h;  isPM = false; }
  else if(h == 12)  { h12 = 12; isPM = true;  }
  else              { h12 = h - 12; isPM = true; }

  first_h  = h12 / 10;
  second_h = h12 % 10;
  first_m  = m / 10;
  second_m = m % 10;

  // Digit 1: suppress leading zero
  if(first_h == 0){
    clearDigit1();
  } else {
    lightDigit1(numbers[first_h]);
  }
  delay(d);

  lightDigit2(numbers[second_h]);  delay(d);
  lightDigit3(numbers[first_m]);   delay(d);
  lightDigit4(numbers[second_m]);  delay(d);
}

void lightSegments(byte number){
  digitalWrite(A, (number >> 6) & 1);
  digitalWrite(B, (number >> 5) & 1);
  digitalWrite(C, (number >> 4) & 1);
  digitalWrite(D, (number >> 3) & 1);
  digitalWrite(E, (number >> 2) & 1);
  digitalWrite(F, (number >> 1) & 1);
  digitalWrite(G, (number >> 0) & 1);
}

void clearDigit1(){
  digitalWrite(CA1, HIGH);
  digitalWrite(CA2, LOW);
  digitalWrite(CA3, LOW);
  digitalWrite(CA4, LOW);
```

```
  lightSegments(blank);
  digitalWrite(DP, isPM ? LOW : HIGH);
}

void lightDigit1(byte number){
  digitalWrite(CA1, HIGH);
  digitalWrite(CA2, LOW);
  digitalWrite(CA3, LOW);
  digitalWrite(CA4, LOW);
  lightSegments(number);
  digitalWrite(DP, isPM ? LOW : HIGH);
}

void lightDigit2(byte number){
  digitalWrite(CA1, LOW);
  digitalWrite(CA2, HIGH);
  digitalWrite(CA3, LOW);
  digitalWrite(CA4, LOW);
  // Blink colon on even seconds
  digitalWrite(DP, (s % 2 == 0) ? LOW : HIGH);
  lightSegments(number);
}

void lightDigit3(byte number){
  digitalWrite(CA1, LOW);
  digitalWrite(CA2, LOW);
  digitalWrite(CA3, HIGH);
  digitalWrite(CA4, LOW);
  digitalWrite(DP, HIGH);
  lightSegments(number);
}

void lightDigit4(byte number){
  digitalWrite(CA1, LOW);
  digitalWrite(CA2, LOW);
  digitalWrite(CA3, LOW);
  digitalWrite(CA4, HIGH);
  digitalWrite(DP, HIGH);
  lightSegments(number);
}
```

Listing 3: Digital Clock — Main Program

7.5. Segment Encoding Reference

The 7-segment displays use **Common Anode** configuration, meaning a segment lights up when its pin is pulled `LOW`. The bit patterns in `numbers[]` follow the order `ABCDEFG` from MSB to LSB:

8. AM/PM Indicator Logic

The decimal point (DP) pin on **Digit 1** is used as the AM/PM indicator. Since the displays are Common Anode, the DP lights up when driven `LOW`.

Digit	Binary Pattern	Hex
0	`1000000`	`0x40`
1	`1111001`	`0x79`
2	`0100100`	`0x24`
3	`0110000`	`0x30`
4	`0011001`	`0x19`
5	`0010010`	`0x12`
6	`0000010`	`0x02`
7	`1111000`	`0x78`
8	`0000000`	`0x00`
9	`0010000`	`0x10`

Table 5: 7-segment digit encoding (Common Anode)

Mode	DP Pin State	DP Visible
PM	`LOW`	✓Yes (lit)
AM	`HIGH`	× No (off)

Table 6: AM/PM indicator behaviour

Digit 2 DP acts as the blinking colon separator. It toggles `LOW/HIGH` based on whether the current second is even or odd.

9. Quick-Start Checklist

- ☐ Wire DS1307 to Arduino (SDA→A4, SCL→A5, VCC→5V, GND→GND)
- ☐ Install **RTClib by Adafruit** via Library Manager
- ☐ Upload **I2C Scanner** — confirm address `0x68` found
- ☐ Edit `hour` and `minute` in **Save Time** code
- ☐ Upload **Save Time** — verify time on Serial Monitor
- ☐ Wire 7-segment displays (segments A–G to pins 2–8, CA1–CA4 to pins 10–13)
- ☐ Upload **Digital Clock** — enjoy your clock!